

How we trained SynACK Decide v7

From an open Gemma 4 model to 59.54 on the Jev Decision Index

59.54

Decision Index 0.2.1, complete run

#1

of 72 on the 2026-09-28 board (PR pending)

THE TASK

A decision model picks an option and says how sure it is

IN: STATE + QUESTIONS

ticket: "I was charged twice
for order #4471 and the app
keeps crashing..."

tier: "pro"

Q1 (choice)

Which team handles it first?

billing · shipping · technical

Q2 (yes/no)

Customer is losing money now.

**OUT: ONE ANSWER PER QUESTION**

team: **billing**

billing 0.91

shipping 0.01

technical 0.08

urgent: **yes**, $p = 0.87$

(probabilities illustrative)

THE SCOREBOARD

The Jev Decision Index: 38 benchmarks, one number

150,759

requests in a complete run

5 areas

knowledge, language, retrieval, tools, arts

Each benchmark is corrected for guessing, so 0 means chance and a skipped question counts as wrong.

Board of 2026-09-28 (index score)

SynACK Decide v7 · 59.54



Jev (TypeSafe) · 57.91



Surogate Rune 26B-A4B v3 · 57.44



Decider chat, Gemma-4-31B · 57.33



Tev1-4B (Together) · 29.2



THE KEY IDEA

We never taught the model to write JSON.

We taught it one thing: the first token of its answer is always one of the option labels A, B, C ... and then it stops.

WHAT IS A TOKEN?

Why every answer is a single token

The model reads text as **tokens**: pieces from a fixed list of 262,144. Each has an ID number.

REAL SPLITS FROM GEMMA'S TOKENIZER

Which team should handle this ticket first ?

a sentence: 8 tokens, mostly whole words

<|turn> <turn|>

a tag: 1 special token each; marks where a turn starts or ends

c

the answer: 1 token

THE ANSWER LABELS

Options are labelled like exam choices, then like spreadsheet columns:

A B C ... Z

AA AB ... AZ

BA ... IV

That is 255 labels, for up to 255 options. Each one is **a single token**, even AA and IV.

So right after the prompt, one step gives a score for every label at once: **the whole answer, with its probabilities, in one step.**

THE TRAINING SHAPE

One example, as the model sees it

```
<|turn>system
Evaluate the supplied decision task. ... Return only its label.<turn|>
<|turn>user
{"state":{"ticket":"I was charged twice ...","customer_tier":"pro"},
"question":"Which team should handle this ticket first?",
"options":{"A":shipping, "B":technical, "C":billing}}<turn|>
<|turn>model
<|channel>thought<channel|> ← prompt: not graded
C<turn|> ← the only graded part
```

Options shuffled in training, so position means nothing

The graded answer is one token: just the label C

Same layout in training and in real use, exactly

IN USE

From label scores to an answer

1

One forward pass over the prompt

2

Read the scores of A, B and C only

3

Turn them into probabilities that add up to 1

4

Map labels back to the option names

```
"team": {"choice": "billing",  
"probabilities": {"billing": 0.91, "shipping": 0.01, "technical": 0.08}},  
"urgent": {"noul": 0.87} ← p(yes); numbers illustrative
```

Always valid output · no text generated · more than 255 options returns "unsupported" instead of a guess

RECAP

In plain words

- 1 Each choice we give the model gets a label: A, B, C and so on. **Each label is a single token.**
- 2 The model reads the question once and **scores every label in one step.**
- 3 We turn those scores into **probabilities that add up to 100%.**
- 4 We return the most likely option, **plus the probability of every option**, so you can see how sure it is.

THE TICKET EXAMPLE

Option	Label	Prob.
shipping	A	0.01
technical	B	0.08
billing	C	0.91

Answer: **billing**, 91% sure.

Probabilities run from 0 to 1, never below zero. Near 0 means unlikely. Numbers illustrative.

HOW LEARNING WORKS • 1

The model scores every possible next token

Token	Score (logit)	Probability
A	1.0	0.058
B	1.5	0.096
C (right)	2.0	0.158
The	3.0	0.429
Billing	2.5	0.260

Gemma 4 knows **262,144 tokens**. For each one it outputs a score.

Softmax turns scores into probabilities: raise e to each score, then divide by the total.

Before training, a chat model wants to start a sentence. Only **15.8%** is on the right label.

Toy example: 5 tokens, made-up scores, exact arithmetic

HOW LEARNING WORKS • 2

Measuring a mistake: cross-entropy

$$\text{loss} = -\log(\text{probability of the right answer})$$

Read it as: how surprised was the model by the right answer? Training makes this small.

$p = 1.0$

0

no mistake

$p = 0.5$

0.69

a coin flip

$p = 0.158$ (our C)

1.85

before training

$p = 0.001$

6.9

confidently wrong

We grade two tokens: the label **C** and the end-of-turn token after it, which teaches the model to stop.

HOW LEARNING WORKS • 3

Distillation: copy a stronger model's confidence

The gold answer only says "**C is right**". A big **teacher** model also knows how plausible A and B were.

So the student also learns to match the teacher's probabilities over the options: **soft labels**.

Safety rule: a teacher is used only when its top choice agrees with the gold answer.

Label	Teacher	Student
A	0.02	0.186
B	0.06	0.307
C	0.92	0.506

$$-(0.02 \cdot \log 0.186 + 0.06 \cdot \log 0.307 + 0.92 \cdot \log 0.506) = 0.730$$

Student: softmax over A, B, C only. Teacher numbers illustrative.

Gradients: which way to move

$\text{gradient} = \text{probability} - \text{target}$

Token	Prob.	Target	Gradient
A	0.058	0	+0.058
B	0.096	0	+0.096
C	0.158	1	-0.842
The	0.429	0	+0.429
Billing	0.260	0	+0.260

+ means "too high, lower it". - means "too low, raise it".

ONE STEP AGAINST THE GRADIENT

Probability of C

0.158 → 0.482

Label loss

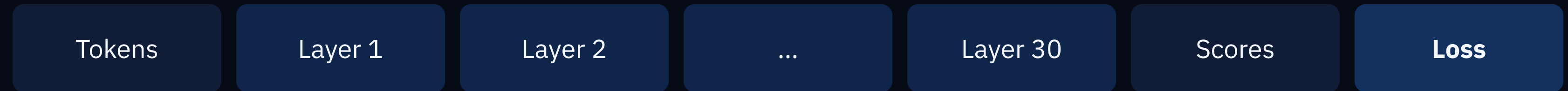
1.847 → 0.729

Teacher loss

0.730 → 0.363

HOW LEARNING WORKS • 5

Backpropagation: share out the blame



→ **Forward pass:** the example flows through all 30 layers to the scores and the loss.

← **Backward pass:** starting from the loss, gradients flow back layer by layer.

The chain rule

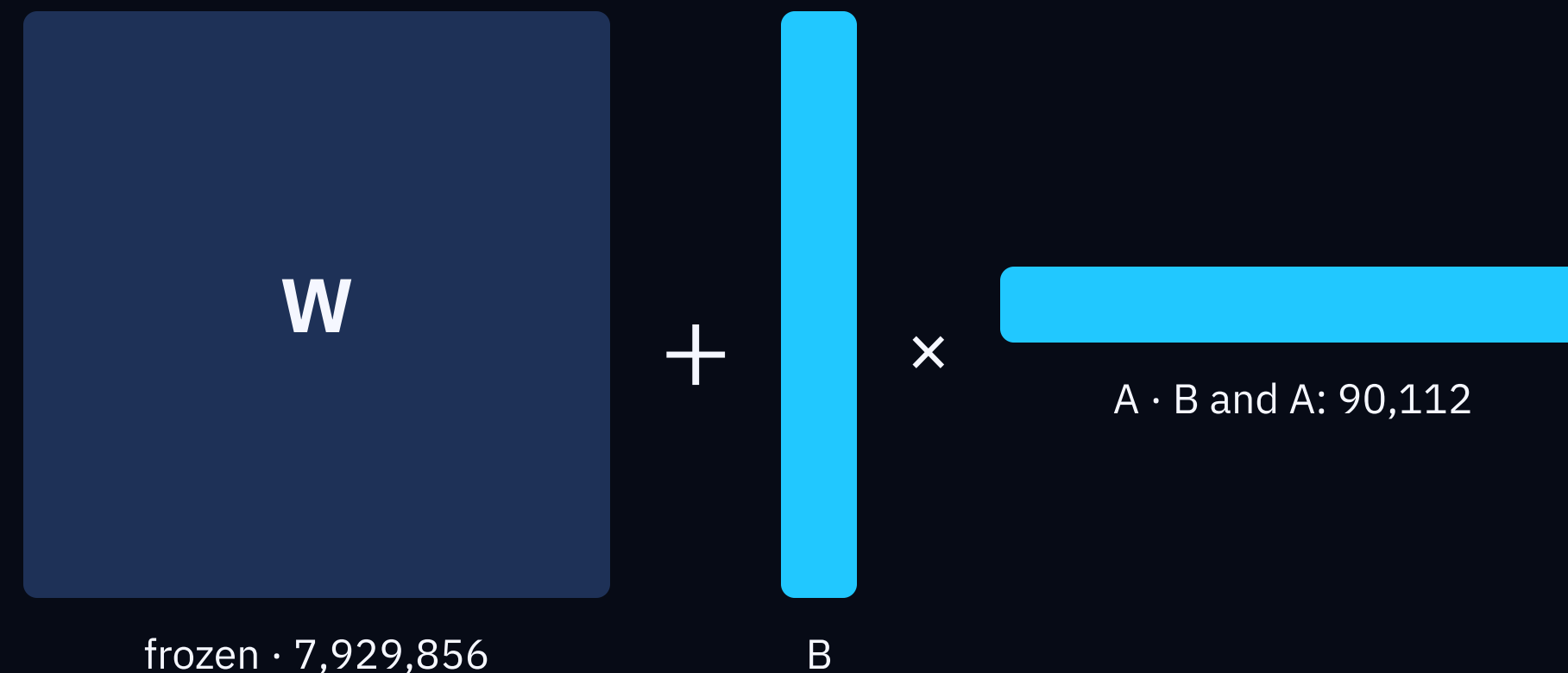
If A affects B and B affects C, then A's effect on C is the two effects multiplied.

The result

Every trainable number learns how much it contributed to the mistake, and in which direction.

HOW LEARNING WORKS • 6

LoRA: train a small add-on



The original weights **never change**. Next to each big matrix sits a thin pair, $B \times A$ (rank 16).

~2%

trainable: about 494M of 26B numbers

B starts at zero, so training starts from the exact Gemma model. At the end $B \times A$ is merged into **W**: same size, same speed.

HOW LEARNING WORKS • 7

The training loop, 3,397 times

1 Batch

32+ examples, grouped by length

2 Forward

scores and loss for each

3 Backward

gradients for the LoRA numbers

4 Safety

clip big gradients; skip a step if loss spikes 8x

5 Update

AdamW nudges each number a tiny step

6 Repeat

one pass over 163,511 examples

Learning rate (step size), peak $3e-5$



Warmup over 169 steps, then cosine decay to zero

8 h 51 min on one 96 GB GPU · no steps skipped · best validation accuracy 79.39%, at the last step

THE PIPELINE

Nine steps from Gemma to 59.54

1 Base model

Gemma 4 26B-A4B, open weights from Google

2 Collect data

public datasets, puzzles, Tev1, GitHub PRs

3 Decontaminate

drop anything that overlaps the test

4 Mix

balance sources by leaderboard area

5 Teacher labels

Kimi-K3 + Gemma-4-31B probabilities

6 Build the set

163,511 training examples

7 Train

fresh LoRA on a rented 96 GB GPU

8 Check

unseen data, probes, PR routing

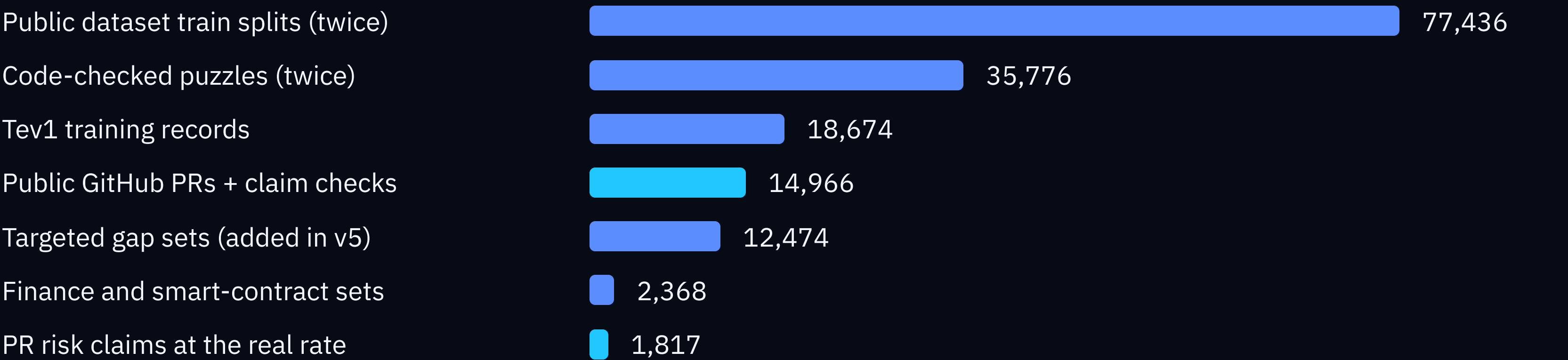
9 Release + run

merge, publish, full run: 59.54

Code for every step: github.com/syn-ack-ai/synack-decide-training (private)

THE DATA

163,511 examples from seven sources



Kept out on purpose: HLE, SATA-Bench, MuSR and SGD, so they stay an honest never-trained yardstick.

THE TEACHERS

Two big models, blended 80/20

Teacher (1,018 held-out questions)	Accuracy
Kimi-K3	80.6%
Gemma-4-31B	77.1%
50/50 blend (v3 to v5)	79.6%
0.8 Kimi + 0.2 Gemma (v6, v7)	81.3%

The blend beat either teacher alone. Where no teacher label existed, the previous release (v5) was the teacher.

HOW WE ASKED

Send the exact training prompt. Allow one answer token. Return the top 20 next tokens with their probabilities.

Keep only the option labels. Reject if under half the probability is on labels.

\$24.80
Kimi-K3, 34,357 q.

\$1.96
Gemma-4-31B

KEEPING THE SCORE HONEST

Judge the model on what it never saw

01 DECONTAMINATE

No test questions in training

A training record is dropped if it shares a 5-word passage with the leaderboard, or 20% of its 13-word runs.

02 NEVER-TRAINED GROUP

Benchmarks with no related data

If a new version only gains where we trained, it learned our data, not the skill.

03 PRIVATE TEST SETS

17 datasets used nowhere else

gen-v1 (2,693 questions) and gen-v2 (1,566), plus PRs from 16 unseen repos and targeted probes.

No benchmaxing. We never add data because a benchmark is low. A version ships only if it is better on what it never saw.

THE JOURNEY

Six versions in five days

Version	What changed	Sample score	Decision
v2	Compact prompt, 255 labels; 12B and MoE trials	53.7 / 54.9	experiments
v3	MoE in full precision on a rented GPU	54.9	first release, replaced
v4	Continued from v3: ordinal and PR-claim fixes	55.7	replaced by v5
v5	Continued from v4: gap sets, self-distillation	58.2	released
v6	Stacked on v5, new teacher blend	58.44	not released: flat, never-trained down
old v7	Fresh LoRA, one pass over 96,534 examples	58.78	not released: lost reasoning
v7	Fresh LoRA, full dose and every fix	61.11	released: official 59.54

Sample score: a 3,000-row estimate of the index (about ± 3 to 4). v7's complete run came in 1.57 below its sample.

THE FAILED RUN

A fresh run must match the dose

v5 had seen its core data **about three times** across stacked rounds.
Old v7 started fresh and saw it **once**.

NEVER-TRAINED BENCHMARKS, OLD V7 VS V5

-14.5

SGD

-13.8

SATA-Bench

-12.1

CRUXEval

-5.2

HLE

It also dropped one fix: PR descriptions claiming "low risk" could sway it again.

WHAT V7 CHANGED

A second pass over every public-dataset
and puzzle row

The balanced PR-claim rows put back
8.5k extra reasoning items

Before any fresh run: count each source's
passes in the model it replaces, and list
every past fix with its test.

THE RESULT

The official run: 59.54

59.54

Decision Index 0.2.1, complete run. Ahead of every entry on the 2026-09-28 board (Jev 57.91).

150,317 requests scored, 0 errors. 167 prompts over the 32,768-token limit counted as wrong. Median 132 ms per request on one GPU.

Area	v7	Best on board	Gap
Knowledge	44.4	51.4	-7.0
Language	63.9	63.5	+0.4
Retrieval	62.4	63.5	-1.1
Tools	79.1	79.3	-0.2
Arts and taste	45.9	41.9	+4.0

"Best on board" is the best model in each area, which differs by area. No single entry leads every area.

THE RELEASE

One model, three formats, public

BF16

NVIDIA GPUs

Full precision; the version that scored 59.54

MLX 8-BIT

Apple Silicon Macs

25 GB; same decision on 97.6% of 500 PRs

GGUF

llama.cpp anywhere

Q8_0 27 GB, Q4_K_M 17 GB; Q8_0 matches on 97.6% of 2,693 questions

RECORDED COSTS FOR THE FINAL MODEL

\$26.76

teacher labels

~\$26.90

v7 training job, 9 h 47 min

~\$26

full run (pre-launch estimate)

LESSONS

What we would tell the next team

01

The answer format was the ceiling

255 single-token labels unlocked 77- and 151-option benchmarks.

02

Distil only where the teacher is right

An 80/20 blend of two teachers beat either one.

03

Judge on what it never saw

v6 rose on the sample and still was not released.

04

A fresh run must match the dose

Count passes per source and carry every past fix.

05

Small steps, with guards

8e-5 collapsed at step ~175; 3e-5 with a spike guard ran clean.

06

The engine changes the answers

vLLM was 4.5x faster but cost about 0.5 index points.

Questions?

Everything here is in the repo, with the code for every step.

CODE + README (PRIVATE)

github.com/syn-ack-ai/synack-decide-training

MODELS (PUBLIC)

huggingface.co/SkyPanther/synack-decide-26b-a4b

LEADERBOARD SUBMISSION

github.com/apolinario/decision-index, PR #71

BEGINNER EXPLAINER

[docs/HOW_TRAINING_WORKS.md](#) in the repo

OFFICIAL RESULTS (PUBLIC)

[SkyPanther/synack-decide-decision-index](#)

WRITE-UP

syn-ack.ai/posts/teaching-a-small-model-to-decide